

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Ярославский государственный университет им. П. Г. Демидова»

Кафедра компьютерных сетей
Направление 01.03.02 «Прикладная математика и информатика»
Профиль «Программирование и технологии искусственного интеллекта»

Сдано на кафедру
« ____ » _____ 2026 г.

Заведующий кафедрой, д. ф.-м. н.,
профессор

_____ С. Д. Глызин

Выпускная квалификационная работа

**Разработка интеллектуального чат-бота для автоматизации онбординга
сотрудников на основе RAG-архитектуры с мультимодальным вводом**

Научный руководитель к. ф.-м. н., доцент

_____ И. В. Парамонов
« ____ » _____ 2026 г.

Студент группы ИВТ-41БО

_____ Б. С. Атрошенко
« ____ » _____ 2026 г.

Ярославль, 2026

АННОТАЦИЯ

Выпускная квалификационная работа посвящена проектированию и реализации интеллектуальной системы поддержки онбординга сотрудников на основе технологии Retrieval-Augmented Generation (RAG). Система принимает запросы в текстовом и голосовом форматах, классифицирует намерение пользователя, выполняет семантический поиск по корпоративной базе знаний и доставляет сформированный ответ в режиме потоковой передачи токенов.

В работе реализованы: RAG-конвейер полного цикла с векторным хранилищем Qdrant; интеллектуальный агент на основе LangGraph с классификацией запросов; асинхронная обработка задач через брокер сообщений RabbitMQ; потоковая доставка ответов по схеме Redis Pub/Sub → Server-Sent Events; модуль распознавания речи на базе faster-whisper; веб-интерфейс на Gradio и административный REST API на FastAPI.

Все инфраструктурные компоненты развёртываются через Docker Compose. Языковая модель, модель векторных представлений и модель распознавания речи работают полностью локально, без обращений к внешним облачным сервисам.

Объём работы: 4 главы, 20 источников, 8 таблиц, 14 рисунков.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
Глава 1. Постановка задачи.....	6
1.1 Описание предметной области	6
1.2 Обзор существующих решений	7
1.3 Цель работы и ожидаемые результаты	9
1.4 Сценарии использования системы.....	9
1.5 Функциональные требования	11
1.6 Нефункциональные требования.....	12
1.7 Задачи разработки	12
Глава 2. Используемые технологии и инструменты	14
2.1 Принцип работы RAG-систем	14
2.2 Оркестрация конвейеров языковых моделей: LangChain и LangGraph.....	15
2.3 Векторная база данных Qdrant.....	16
2.4 Локальный запуск языковой модели: Ollama	17
2.5 Брокер сообщений RabbitMQ	18
2.6 Redis: кэш, хранилище сессий и шина данных	18
2.7 Автоматическое распознавание речи: faster-whisper	19
2.8 Серверная и клиентская части: FastAPI и Gradio	20
Глава 3. Реализация системы.....	21
3.1 Разработка RAG-конвейера.....	22
3.1.1 Архитектура конвейера загрузки и индексации документов.....	23
3.1.2 Стратегии сегментации текста: сравнительный анализ	24
3.1.3 Реализованная схема сегментации.....	25
3.1.4 Построение векторного хранилища	26
3.1.5 Алгоритм семантического поиска.....	27
3.2 Интеллектуальный агент на основе LangGraph	28
3.2.1 Архитектура агента.....	28
3.2.2 Описание графа состояний.....	29
3.2.3 Узел классификации намерений	29
3.2.4 Узел ответов по базе знаний.....	30
3.2.5 Узел обработки обычных запросов.....	31
3.2.6 Управление диалоговой памятью	31
3.3 Асинхронная обработка задач с использованием RabbitMQ.....	32

3.3.1 Обоснование применения брокера сообщений	33
3.3.2 Топология обмена сообщениями	34
3.3.3 Публикатор задач.....	35
3.3.4 Потребитель сообщений и обработчики задач	35
3.4 Поточковая передача ответов: Redis Pub/Sub и SSE.....	36
3.4.1 Архитектурное решение потоковой передачи.....	36
3.4.2 Реализация Redis Pub/Sub для доставки токенов.....	37
3.4.3 Server-Sent Events в FastAPI	37
3.5 Мультимодальный ввод: обработка голосовых запросов.....	38
3.5.1 Обоснование выбора движка распознавания речи	39
3.5.2 Реализация модуля транскрибации	40
3.5.3 Интеграция распознавания речи в общий конвейер.....	40
3.6 Пользовательский интерфейс и управление базой знаний.....	41
3.6.1 Интерфейс чата	41
3.6.2 Модуль управления документами.....	42
3.6.3 REST API администрирования.....	42
3.7 Оценка работоспособности системы	43
3.7.1 Методика оценки	43
3.7.2 Оценка качества ответов RAG-конвейера	44
3.7.3 Оценка производительности	45
3.7.4 Результаты и обсуждение	45
Глава 4. Конфигурирование и развёртывание системы	47
4.1 Управление конфигурацией.....	47
4.2 Жизненный цикл FastAPI-приложения	47
4.3 Контейнеризация с Docker Compose	48
ЗАКЛЮЧЕНИЕ.....	50
СПИСОК ЛИТЕРАТУРЫ	53

ВВЕДЕНИЕ

Процесс адаптации новых сотрудников в компании, называемый онбордингом (от англ. onboarding — введение в должность), представляет собой одну из ключевых задач управления человеческими ресурсами. Современные компании располагают обширными корпоративными базами знаний, включающими регламенты, политики, инструкции по внутренним инструментам и описания бизнес-процессов, однако разрозненность этих документов существенно затрудняет эффективное использование накопленного опыта.

Появление больших языковых моделей (Large Language Models, LLM) открыло возможности для создания интеллектуальных ассистентов, способных предоставлять ответы на вопросы о корпоративных процессах в режиме естественного диалога. Однако применение LLM без дополнительных механизмов привязки к конкретной базе знаний сопряжено с проблемой «галлюцинаций» — генерацией фактически неверных, но правдоподобно звучащих утверждений. Технология Retrieval-Augmented Generation (RAG), впервые формализованная в работе Lewis et al. [1], решает эту проблему: при обработке каждого запроса система извлекает из документального хранилища наиболее релевантные фрагменты и передаёт их языковой модели в качестве контекста.

Работа посвящена проектированию и реализации системы поддержки сотрудников, получившей рабочее название FinBridge RAG Assistant. Разработанная система принимает запросы в текстовом и голосовом форматах, классифицирует намерение запроса, извлекает релевантные фрагменты из корпоративной базы знаний, формирует ответ с помощью локально развёрнутой языковой модели и доставляет его пользователю в режиме потоковой передачи токенов, создавая эффект «живого» печатающего ответа.

Одним из ключевых архитектурных решений системы является применение асинхронного брокера сообщений RabbitMQ, позволяющего

немедленно возвращать управление клиенту при получении запроса и обрабатывать вычислительно ёмкие операции в фоновом режиме. Для доставки потока токенов от фонового обработчика к браузеру применяется связка Redis Pub/Sub и Server-Sent Events (SSE). Персистентная память диалога поддерживается через Redis с применением прогрессивной суммаризации.

Актуальность работы определяется совокупностью практических и технических факторов. Задача автоматизации онбординга востребована широким кругом организаций, не располагающих бюджетом на приобретение коммерческих чат-ботов корпоративного уровня. Реализованная архитектура целиком построена на открытом программном обеспечении: языковая модель, векторная база данных и модель распознавания речи запускаются локально и не требуют лицензионных затрат.

Практическим результатом работы является полностью функционирующая система, развёрнутая в окружении Docker, с веб-интерфейсом на основе Gradio и REST API на FastAPI. Функциональность продемонстрирована на синтетически подготовленном наборе корпоративных документов компании FinBridge в формате Markdown.

Глава 1. Постановка задачи

1.1 Описание предметной области

Онбординг представляет собой организованный процесс введения нового сотрудника в рабочий контекст компании: знакомство с корпоративной культурой, регламентами, внутренними инструментами, командными процессами и бизнес-направлениями. В крупных организациях накопленные знания распределены по десяткам документов, вики-страниц и обучающих материалов. Для нового сотрудника ориентация в этом массиве информации требует значительных временных затрат и неизбежно порождает поток вопросов к коллегам.

Типичные проблемы традиционного онбординга включают следующие. Во-первых, сотрудники вынуждены прерывать рабочий процесс коллег рутинными вопросами, ответы на которые зафиксированы в документации, но труднодоступны для быстрого поиска. Во-вторых, корпоративный поиск по вики-системам требует точного знания терминологии и местонахождения документа — навыки, которых у новичка нет. В-третьих, качество передаваемых знаний неравномерно: каждый наставник расставляет акценты по-своему, что приводит к противоречивым представлениям о регламентах. Наконец, голосовая коммуникация всё шире используется в удалённых командах, однако корпоративные базы знаний голосовой ввод не поддерживают.

Интеллектуальный чат-бот с RAG-архитектурой устраняет описанные проблемы системным образом: система доступна круглосуточно, предоставляет единообразные ответы с указанием источника, принимает как текстовые, так и голосовые запросы и адаптирует ответы к контексту текущего диалога благодаря механизму памяти сессии.

Предметная область применения системы — финансовая компания условного профиля FinBridge. Для демонстрации функциональности был сформирован набор синтетических корпоративных документов в формате

Markdown, охватывающий типичные темы онбординга: политику безопасности, процедуры согласования, описание внутренних инструментов и кадровые регламенты.

1.2 Обзор существующих решений

Задача автоматизации онбординга с помощью интеллектуальных ассистентов в настоящее время решается рядом коммерческих продуктов. Прямых аналогов разрабатываемой системы — полностью локального RAG-ассистента с поддержкой голосового ввода — на рынке немного, однако существует широкий круг косвенных конкурентов, решающих смежную задачу предоставления сотрудникам ответов по корпоративной базе знаний. Для обоснования новизны разрабатываемого решения ниже проанализированы наиболее представительные из них.

Платформы корпоративного поиска Glean и Guru объединяют документы из множества источников — корпоративных вики, мессенджеров, облачных хранилищ — и предоставляют ответы на естественном языке с указанием источника. Glean делает акцент на сквозном поиске по всем системам компании, Guru — на «проверенных» ответах, формируемых только из материалов, подтверждённых экспертами. Обе платформы являются облачными сервисами с подписочной моделью корпоративного уровня и не предусматривают развёртывания в собственном контуре организации.

Решения ServiceNow Virtual Agent, Leena AI и Moveworks представляют класс HR- и ITSM-ориентированных виртуальных агентов, автоматизирующих обработку типовых обращений сотрудников — вопросов по кадровым политикам, льготам, ИТ-поддержке. Эти продукты тесно интегрированы с соответствующими корпоративными платформами, ориентированы на крупные предприятия и распространяются по модели дорогостоящей корпоративной лицензии.

На российском рынке схожую задачу решают облачные сервисы TEAMLY AI, Wikilect и аналогичные платформы: они индексируют

корпоративную документацию, строят векторную базу и отвечают на вопросы сотрудников. По опубликованным данным, внедрение подобных решений ускоряет адаптацию новых сотрудников и сокращает время поиска информации, однако базовая стоимость внедрения исчисляется миллионами рублей, а данные обрабатываются на стороне поставщика.

Сравнение перечисленных решений с разрабатываемой системой по ключевым для поставленной задачи признакам приведено в таблице 1.

Таблица 1 — Сравнение существующих решений с разрабатываемой системой

Решение	Развёртывание	Стоимость владения	Голосовой ввод	Открытый код
Glean, Guru	Облако (SaaS)	Подписка корпоративного уровня	Нет	Нет
ServiceNow Virtual Agent	Облако (SaaS)	Корпоративная лицензия	Ограниченно	Нет
Leena AI, Moveworks	Облако (SaaS)	Корпоративная лицензия	Нет	Нет
TEAMLY AI, Wikilect	Облако (SaaS)	От нескольких млн руб.	Нет	Нет
Разрабатываемая система	Локально, в контуре организации	Только стоимость инфраструктуры	Да	Да

Анализ показывает, что все рассмотренные решения являются облачными сервисами с подписной или лицензионной моделью: корпоративные документы передаются и обрабатываются на стороне поставщика, а стоимость владения существенна. Кроме того, подавляющее большинство из них ограничено текстовым каналом ввода. Разрабатываемая система обладает совокупностью отличительных признаков, отсутствующих у рассмотренных конкурентов: полностью локальное исполнение всех компонентов, включая языковую модель и модуль распознавания речи, что исключает выход корпоративных данных за пределы инфраструктуры и не требует подключения к сети Интернет; построение исключительно на

открытом программном обеспечении, сводящее лицензионные затраты к нулю; встроенная поддержка голосового ввода; прозрачная и расширяемая архитектура, допускающая адаптацию под конкретную организацию. Именно сочетание локальности, нулевой стоимости лицензий и мультимодального ввода составляет новизну предлагаемого решения.

1.3 Цель работы и ожидаемые результаты

Цель настоящей работы — разработать и реализовать интеллектуальную систему поддержки онбординга сотрудников, которая принимает запросы в текстовом и голосовом форматах, формирует точные ответы по корпоративной базе знаний с использованием RAG-архитектуры и доставляет их пользователю в режиме потоковой передачи токенов.

Ожидаемые результаты работы:

1. Реализованный RAG-конвейер полного цикла: загрузка документов, их сегментация, вычисление векторных представлений, индексация в векторной базе данных, семантический поиск и генерация ответа.
2. Интеллектуальный агент на основе LangGraph с классификацией намерений и двумя специализированными режимами обработки запросов.
3. Асинхронная архитектура обработки задач на основе брокера сообщений RabbitMQ, обеспечивающая немедленный отклик API.
4. Механизм потоковой доставки ответов через Redis Pub/Sub и Server-Sent Events.
5. Модуль мультимодального ввода с автоматическим распознаванием речи.
6. Веб-интерфейс пользователя и административный модуль управления базой знаний.
7. Среда развёртывания на Docker Compose с полной инфраструктурой.

1.4 Сценарии использования системы

Система предусматривает две роли пользователей: сотрудник — конечный пользователь чат-бота — и администратор базы знаний —

специалист, ответственный за актуальность документов. Ниже приведены сценарии использования для каждой роли.

Роль «Сотрудник».

Сценарий использования 1 — текстовый запрос к базе знаний. При первой загрузке страницы система автоматически генерирует уникальный идентификатор сессии. Сотрудник вводит вопрос и нажимает «Отправить»; система регистрирует задачу и немедленно возвращает её идентификатор. Интерфейс устанавливает SSE-соединение и начинает получать токены ответа. По завершении генерации к ответу прикрепляются ссылки на документы-источники. История диалога сохраняется в Redis.

Сценарий использования 2 — голосовой запрос. Сотрудник нажимает «Записать» и задаёт вопрос голосом; запись автоматически останавливается при завершении речи. Система транскрибирует запись и отображает расшифровку перед началом генерации ответа. Дальнейший поток обработки идентичен сценарию использования 1.

Сценарий использования 3 — обычная беседа. Если запрос не связан с корпоративной базой знаний, система генерирует ответ напрямую через языковую модель без обращения к векторному хранилищу.

Роль «Администратор базы знаний».

Сценарий использования 4 — загрузка документа. Администратор выбирает файл Markdown; система проверяет расширение и сохраняет файл в директорию документов на сервере.

Сценарий использования 5 — индексация документа. Администратор выбирает документы в таблице и нажимает «Индексировать»; система сегментирует тексты, вычисляет векторные представления и загружает фрагменты в Qdrant.

Сценарий использования 6 — удаление документа. Система удаляет соответствующие векторы из Qdrant и файлы с диска, возвращая статистику удалённых записей.

Сценарий использования 7 — переиндексация документа. При обновлении содержимого документа администратор откатывает индексацию, а затем повторно индексирует документ.

1.5 Функциональные требования

На основании анализа предметной области и сценариев использования сформулированы следующие функциональные требования к системе.

ФТ-1. Система должна принимать текстовые запросы через HTTP POST-метод и возвращать идентификатор задачи немедленно, без ожидания завершения генерации ответа.

ФТ-2. Система должна принимать аудиофайлы с голосовыми запросами через отдельный HTTP POST-метод и автоматически их транскрибировать.

ФТ-3. Система должна классифицировать каждый входящий запрос как обращение к базе знаний (knowledge_base) или как обычную беседу (small_talk).

ФТ-4. При обработке запроса к базе знаний система должна извлекать семантически близкие фрагменты документов и использовать их как контекст при генерации ответа.

ФТ-5. Система должна доставлять ответ пользователю в режиме потоковой передачи токенов через SSE-соединение.

ФТ-6. При обработке голосового запроса система должна публиковать транскрипцию как отдельное SSE-событие до начала потоковой передачи токенов.

ФТ-7. Система должна сохранять историю диалога в рамках сессии и учитывать не менее последних шести сообщений при формировании запроса к языковой модели.

ФТ-8. Система должна поддерживать загрузку документов Markdown через веб-интерфейс администратора.

ФТ-9. Система должна обеспечивать индексацию, переиндексацию и удаление документов из векторного хранилища.

ФТ-10. Ответ на запрос к базе знаний должен содержать ссылки на документы-источники.

1.6 Нефункциональные требования

НФТ-1. Отзывчивость API. Методы регистрации задач должны возвращать ответ немедленно, не ожидая завершения обработки.

НФТ-2. Неблокирующий событийный цикл. Синхронные вычислительно ёмкие операции (транскрибация речи) должны выполняться в пуле потоков, не блокируя событийный цикл `asyncio`.

НФТ-3. Изолированность сессий. Данные диалога каждого пользователя хранятся в изолированном пространстве ключей Redis с временем жизни не менее одного часа.

НФТ-4. Локальность вычислений. Языковая модель, модель векторных представлений и модель распознавания речи работают полностью локально, без обращений к внешним платным сервисам.

НФТ-5. Контейнеризация. Все инфраструктурные компоненты запускаются единой командой `docker compose up -d`.

НФТ-6. Расширяемость базы знаний. Добавление новых документов выполняется без перезапуска приложения.

НФТ-7. Документация API. Для всех HTTP-методов автоматически генерируется интерактивная документация Swagger UI.

1.7 Задачи разработки

Для достижения поставленной цели необходимо выполнить следующее.

1. Разработать RAG-конвейер: реализовать загрузку, сегментацию документов и их индексацию в векторную базу данных Qdrant с поддержкой семантического поиска.

2. Реализовать интеллектуальный агент на основе LangGraph с классификацией намерений и двумя режимами обработки запросов (RAG и обычная беседа), а также с механизмом персистентной памяти диалога.

3. Реализовать асинхронную систему обработки задач с использованием брокера сообщений RabbitMQ, обеспечивающую немедленный отклик HTTP API.

4. Реализовать механизм потоковой доставки токенов от фонового обработчика к клиентскому браузеру на основе Redis Pub/Sub и Server-Sent Events.

5. Интегрировать модуль автоматического распознавания речи на базе faster-whisper с корректным переносом синхронных операций в отдельный поток исполнения.

6. Разработать пользовательский веб-интерфейс на Gradio и административный модуль управления базой знаний с соответствующим REST API на FastAPI.

Глава 2. Используемые технологии и инструменты

2.1 Принцип работы RAG-систем

Retrieval-Augmented Generation (RAG) — архитектурный подход к созданию систем генерации текста, в котором запрос пользователя обогащается динамически извлечёнными фрагментами внешней базы знаний перед передачей языковой модели. Метод был впервые формализован в работе Lewis et al. [1] и в настоящее время является стандартом построения корпоративных вопросно-ответных систем.

Архитектура RAG-системы включает две независимые фазы.

Фаза индексации выполняется однократно или при обновлении документации: исходные тексты загружаются, разбиваются на фрагменты, для каждого фрагмента вычисляется векторное представление (эмбединг), и полученные векторы вместе с текстом сохраняются в векторной базе данных.

Фаза запроса выполняется при каждом обращении пользователя: вопрос преобразуется в вектор, выполняется поиск ближайших соседей, найденные фрагменты объединяются с вопросом в единый запрос и передаются языковой модели.

Ключевым преимуществом RAG является фактологическая обоснованность ответов: языковая модель получает явные цитаты из корпоративных документов и опирается на них при генерации, что кардинально снижает частоту галлюцинаций. Дополнительным достоинством является динамическая актуализация базы знаний: переобучение модели не требуется — достаточно переиндексировать обновлённые документы. В настоящей работе три роли RAG-системы выполняют модель векторных представлений BAAI/bge-small-en, векторная база данных Qdrant и языковая модель phi3:mini, запускаемая через Ollama.

Обобщённая схема работы RAG-системы, объединяющая обе фазы, приведена на рис. 1. На схеме показано, как при индексации документы преобразуются в векторы и сохраняются в базе данных, а при обработке

запроса вопрос пользователя сопоставляется с этими векторами, после чего найденный контекст вместе с вопросом передаётся языковой модели для генерации итогового ответа.

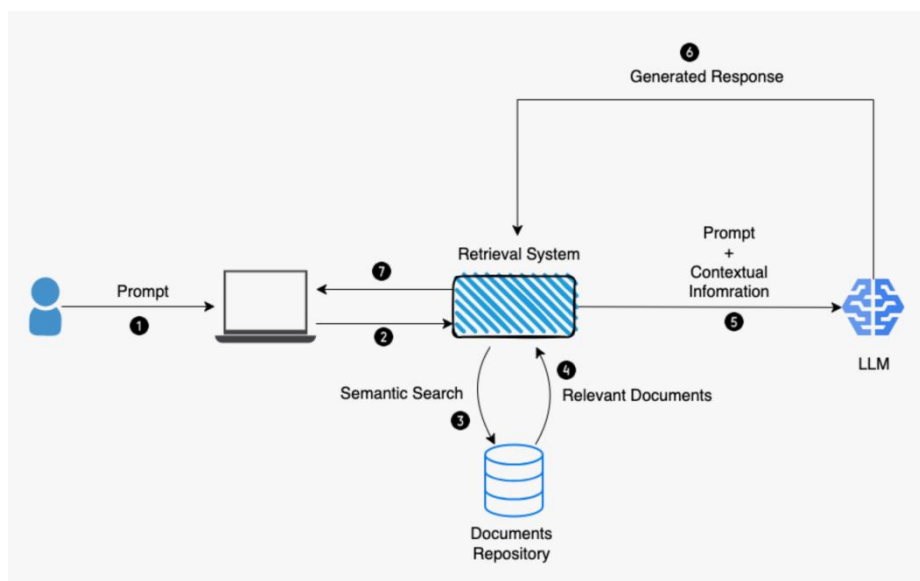


Рис. 1 — Упрощённая схема работы RAG-системы

2.2 Оркестрация конвейеров языковых моделей: LangChain и LangGraph

LangChain [2] — открытая библиотека языка Python для построения приложений на основе языковых моделей, предоставляющая унифицированные абстракции для работы с моделями, шаблонами запросов, цепочками вызовов и векторными хранилищами. Центральной абстракцией является интерфейс Runnable, позволяющий компоновать компоненты в конвейеры с помощью оператора | (LangChain Expression Language, LCEL).

LangGraph [3] — расширение LangChain для построения агентов в форме ориентированных графов состояний. В отличие от линейных цепочек LCEL, LangGraph поддерживает ветвление, условные переходы и сохранение состояния между шагами. Агент описывается как граф состояний StateGraph: разработчик определяет схему состояния (модель Pydantic), набор узлов (функции или корутины Python) и условные рёбра между ними.

Существенным преимуществом LangGraph для потоковых сценариев является метод `astream_events`: он позволяет подписаться на низкоуровневые

события выполнения графа, включая получение токена от языковой модели и завершение узла, что даёт возможность публиковать каждый токен по мере его генерации. В качестве альтернативы рассматривался фреймворк AutoGen от Microsoft, однако он ориентирован на многоагентные сценарии и скрывает детали управления потоком, что неприемлемо для задачи с двусторонним ветвлением.

Сопоставление двух подходов к построению приложений показано на рис. 2: линейная цепочка LangChain противопоставляется графу состояний LangGraph, в котором обработка может ветвиться в зависимости от промежуточного результата классификации запроса.

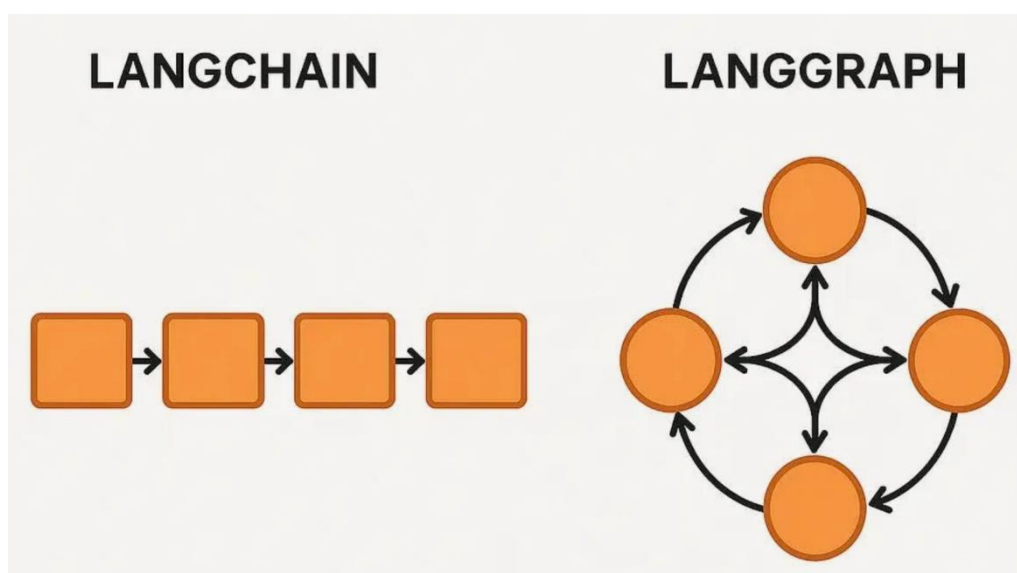


Рис. 2 — Сравнение подходов LangChain и LangGraph

2.3 Векторная база данных Qdrant

Qdrant [4] — специализированная база данных для хранения и поиска по высокоразмерным векторам, написанная на языке Rust. Система реализует алгоритм приближённого поиска ближайших соседей HNSW (Hierarchical Navigable Small World) и поддерживает косинусную, евклидову метрики и скалярное произведение. Qdrant предоставляет развитый механизм фильтрации по произвольным полям полезной нагрузки, что критично для адресного удаления фрагментов конкретного документа.

По сравнению с альтернативами Qdrant обладает рядом преимуществ: Pinecone требует облачного аккаунта; ChromaDB изначально проектировался

как встраиваемая база без полноценного клиент-серверного режима; Weaviate имеет более высокие накладные расходы при развёртывании в Docker. Qdrant является полностью открытым, поддерживает персистентность на диске и используется через официальный клиент qdrant-client и высокоуровневую обёртку QdrantVectorStore из библиотеки langchain-qdrant.

2.4 Локальный запуск языковой модели: Ollama

Ollama [5] — платформа для локального запуска языковых моделей, предоставляющая унифицированный REST API, совместимый с интерфейсом OpenAI Chat Completions. Ollama берёт на себя загрузку весов модели в формате GGUF, управление квантизацией и распределение вычислений между центральным и графическим процессорами. В работе применяется модель phi3:mini — компактная квантизованная модель с 3,8 млрд параметров, функционирующая на центральном процессоре без требований к графическому ускорителю.

Схема взаимодействия Ollama с библиотекой LangChain приведена на рис. 3: LangChain обращается к локальному REST API Ollama, который, в свою очередь, выполняет вывод языковой модели на доступном вычислительном устройстве и возвращает результат в потоковом режиме.

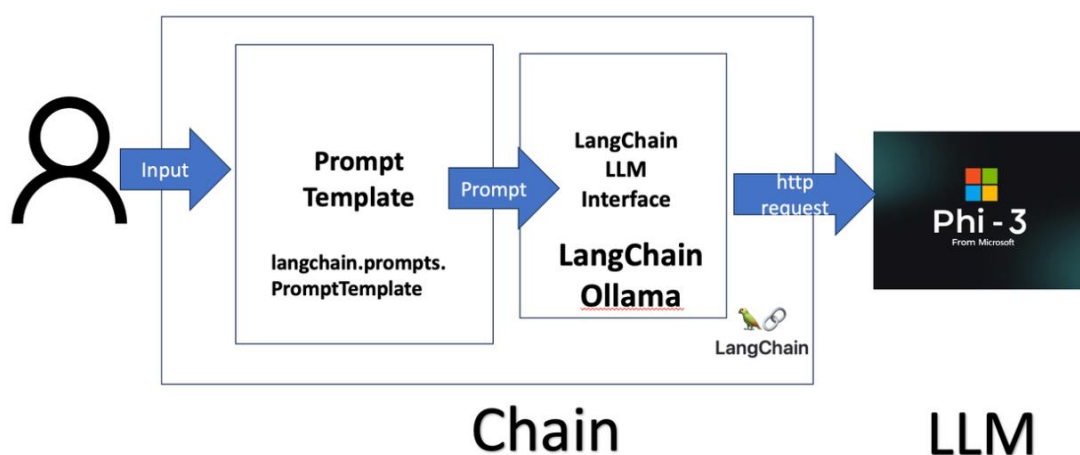


Рис. 3 — Взаимодействие Ollama с библиотекой LangChain

Применение локально работающей модели обусловлено требованием конфиденциальности: корпоративные документы не должны покидать

периметр инфраструктуры. Локальный запуск также исключает операционные затраты на токены и обеспечивает работоспособность системы без подключения к сети Интернет.

2.5 Брокер сообщений RabbitMQ

RabbitMQ [6] — брокер сообщений с открытым исходным кодом, реализующий протокол AMQP. Взаимодействие организовано по модели «точка обмена — очередь — потребитель»: публикатор направляет сообщение в точку обмена с указанием ключа маршрутизации; точка обмена доставляет его в очередь согласно правилам связки; потребитель извлекает сообщения и отправляет подтверждение после успешной обработки.

Использование брокера сообщений решает проблему длительного HTTP-запроса: генерация ответа языковой моделью занимает от нескольких секунд до нескольких десятков секунд, и ожидание в открытом HTTP-соединении неприемлемо. Разделение фаз «принятие задачи» (мгновенный POST-запрос) и «доставка результата» (асинхронный поток SSE) устраняет эту проблему принципиально. Среди рассмотренных вариантов RabbitMQ в связке с библиотекой `aiorika` признан оптимальным: он обеспечивает нативную интеграцию с `asyncio`, гибкую маршрутизацию и удобный веб-интерфейс управления.

2.6 Redis: кэш, хранилище сессий и шина данных

Redis [7] — высокопроизводительное хранилище данных в оперативной памяти. В архитектуре системы Redis выполняет три независимые роли.

Первая роль — хранилище истории диалога: компонент `RedisChatMessageHistory` из библиотеки `langchain-community` сериализует объекты сообщений пользователя и ассистента в список Redis под ключом, привязанным к идентификатору сессии.

Вторая роль — хранилище прогрессивных суммаризаций: после каждого хода диалога вычисляется краткое резюме и сохраняется в Redis под

отдельным ключом, что предотвращает переполнение контекстного окна языковой модели.

Схема использования каналов Redis Pub/Sub приведена на рис. 4: фоновый обработчик публикует токены в канал, привязанный к идентификатору задачи, а SSE-метод подписывается на этот канал и транслирует полученные события браузеру пользователя.

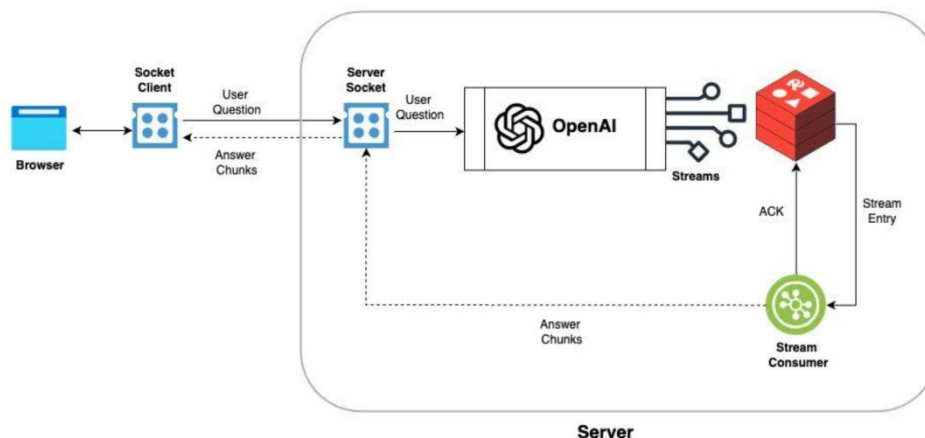


Рис. 4 — Использование каналов Redis Pub/Sub

Третья роль — канал Pub/Sub для передачи токенов: фоновый обработчик публикует токены в канал `llm:result:{task_id}`, а SSE-метод подписывается на него и транслирует события браузеру.

2.7 Автоматическое распознавание речи: faster-whisper

Faster-whisper [8] — оптимизированная реализация архитектуры Whisper от OpenAI, использующая библиотеку CTranslate2 для ускоренного вывода с квантизацией int8. По сравнению с оригинальной реализацией openai-whisper на PyTorch faster-whisper демонстрирует в 2–4 раза более высокую скорость транскрибации на центральном процессоре при сопоставимом качестве. Модель поддерживает автоматическое определение языка среди 99 поддерживаемых языков. В работе по умолчанию применяется модель base (74 МБ), обеспечивающая баланс между скоростью (2–5 секунд на вопрос длительностью 10–15 секунд) и точностью.

2.8 Серверная и клиентская части: FastAPI и Gradio

FastAPI [9] — современный асинхронный веб-фреймворк языка Python, построенный на основе Starlette и Pydantic. Он обеспечивает автоматическую валидацию данных, генерацию интерактивной документации Swagger UI и нативную поддержку потоковых ответов StreamingResponse для Server-Sent Events. Асинхронная модель позволяет обслуживать тысячи конкурентных SSE-соединений без выделения отдельного потока на каждое соединение.

Gradio [10] — библиотека языка Python для создания веб-интерфейсов систем машинного обучения. Она предоставляет готовые компоненты — поле чата, виджет записи звука, интерактивную таблицу — при минимальном объёме кода. Для потоковых обновлений Gradio поддерживает генераторные обработчики: функция-обработчик может последовательно возвращать промежуточные состояния компонента. Двухвкладочный интерфейс системы реализован в двух файлах без использования HTML, CSS и JavaScript.

Внешний вид административного интерфейса, реализованного средствами Gradio, показан на рис. 5: он включает интерактивную таблицу документов с их характеристиками (имя, размер, число фрагментов, статус индексации) и элементы управления операциями индексации и удаления.

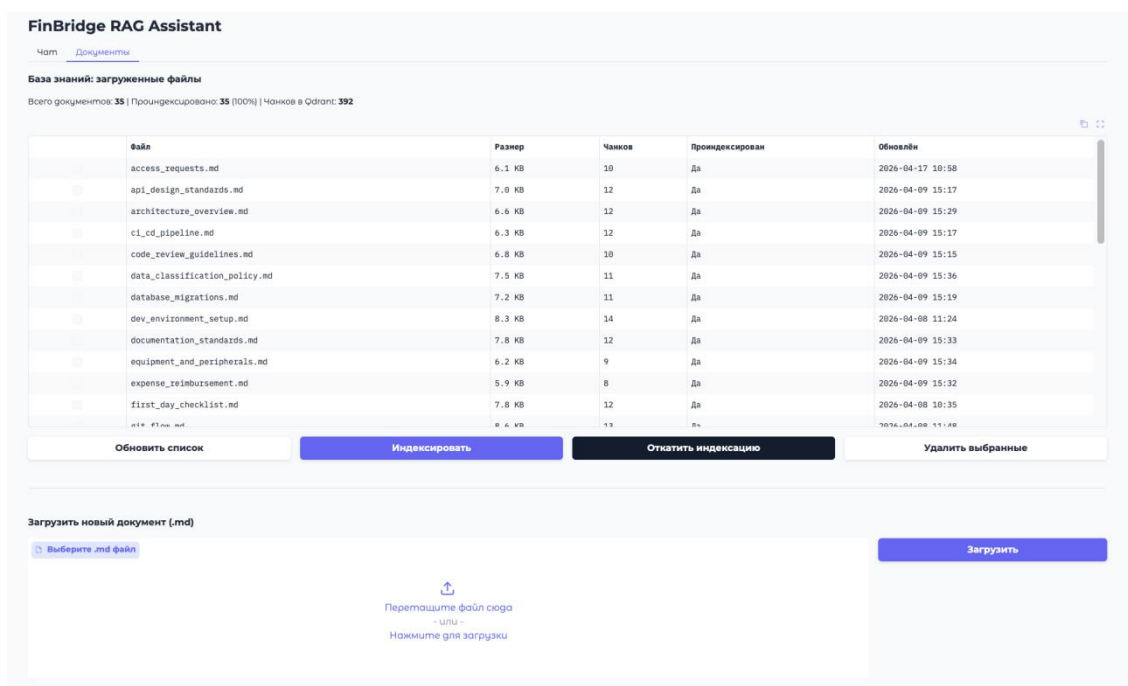


Рис. 5 — Административный интерфейс на основе Gradio

Глава 3. Реализация системы

Настоящая глава посвящена реализации системы. В ней последовательно описываются все программные модули — RAG-конвейер, интеллектуальный агент, подсистема асинхронной обработки задач, механизм потоковой передачи ответов, модуль мультимодального ввода и пользовательский интерфейс, — а также приводится оценка работоспособности готовой системы. Чтобы обеспечить возможность проследить, где именно описана реализация каждого функционального и нефункционального требования и каждого сценария использования, в таблице 2 приведено их соответствие разделам настоящей главы.

Таблица 2 — Соответствие требований и сценариев использования разделам реализации

Требование / сценарий использования	Раздел(ы), где описана реализация
ФТ-1. Немедленный возврат идентификатора задачи	3.3.1, 3.3.3, 3.3.4
ФТ-2. Приём и транскрибация аудиофайлов	3.5.2, 3.5.3
ФТ-3. Классификация запроса	3.2.3
ФТ-4. Извлечение фрагментов как контекста	3.1.5, 3.2.4
ФТ-5. Потокковая передача токенов через SSE	3.4.1, 3.4.3
ФТ-6. Публикация транскрипции отдельным событием	3.4.1, 3.5.3
ФТ-7. История диалога (не менее шести сообщений)	3.2.6
ФТ-8. Загрузка документов Markdown	3.6.2, 3.6.3
ФТ-9. Индексация, переиндексация и удаление документов	3.1.1, 3.6.3
ФТ-10. Ссылки на документы-источники в ответе	3.1.5, 3.2.4
НФТ-1. Отзывчивость API	3.3.1, 3.3.3, 3.7.3

Требование / сценарий использования	Раздел(ы), где описана реализация
НФТ-2. Неблокирующий событийный цикл	3.5.3
НФТ-3. Изолированность сессий в Redis	3.2.6
НФТ-4. Локальность вычислений	2.4, 3.5.1
НФТ-5. Контейнеризация	4.3
НФТ-6. Расширяемость базы знаний	3.6.2, 3.6.3
НФТ-7. Документация API (Swagger UI)	3.6.3
Сценарий использования 1. Текстовый запрос	3.2, 3.3, 3.4, 3.6.1
Сценарий использования 2. Голосовой запрос	3.5, 3.6.1
Сценарий использования 3. Обычная беседа	3.2.5
Сценарий использования 4. Загрузка документа	3.6.2, 3.6.3
Сценарий использования 5. Индексация документа	3.1.1, 3.6.3
Сценарий использования 6. Удаление документа	3.1.1, 3.6.3
Сценарий использования 7. Переиндексация документа	3.1.1, 3.6.3

3.1 Разработка RAG-конвейера

RAG-конвейер — основа всей системы: именно он наполняет векторное хранилище фрагментами корпоративных документов и извлекает из них контекст, на который опирается языковая модель при генерации ответа. Без этого модуля система не смогла бы давать обоснованные ответы по базе знаний. В настоящем разделе описываются архитектура конвейера, выбор стратегии сегментации текста, построение векторного хранилища и алгоритм семантического поиска.

3.1.1 Архитектура конвейера загрузки и индексации документов

RAG-конвейер системы организован как двухфазный процесс с чётким разделением ответственности. Фаза индексации инициируется явно через административный API и обрабатывает документы из директории `finbridge/docs/`. Фаза поиска вызывается автоматически при каждом пользовательском запросе, классифицированном как обращение к базе знаний.

Вся логика индексирующей фазы инкапсулирована в классе `DocsDirectoryIngestion` (модуль `service/RAG/docs_service.py`). Класс реализует паттерн статических методов-сервисов и предоставляет интерфейс: `docs_load` — загрузка и индексация; `erase_docs` — удаление из Qdrant и файловой системы; `get_documents_info` — статистика индексации; `rm_points_by_source` — адресное удаление векторов конкретного документа.

Обработка одного документа проходит четыре стадии: загрузка текста из файла `.md` через загрузчик `TextLoader`; разбиение на фрагменты через `RecursiveCharacterTextSplitter`; вычисление векторного представления каждого фрагмента моделью `BAAI/bge-small-en`; пакетная загрузка фрагментов в Qdrant. Каждый фрагмент сохраняется с полезной нагрузкой, содержащей полный путь к файлу-источнику, что позволяет при удалении документа точно удалять только его векторы по фильтру.

Последовательность стадий обработки документа в фазе индексации приведена на рис. 6: исходный файл проходит загрузку, сегментацию, вычисление векторных представлений и сохранение в векторной базе данных.

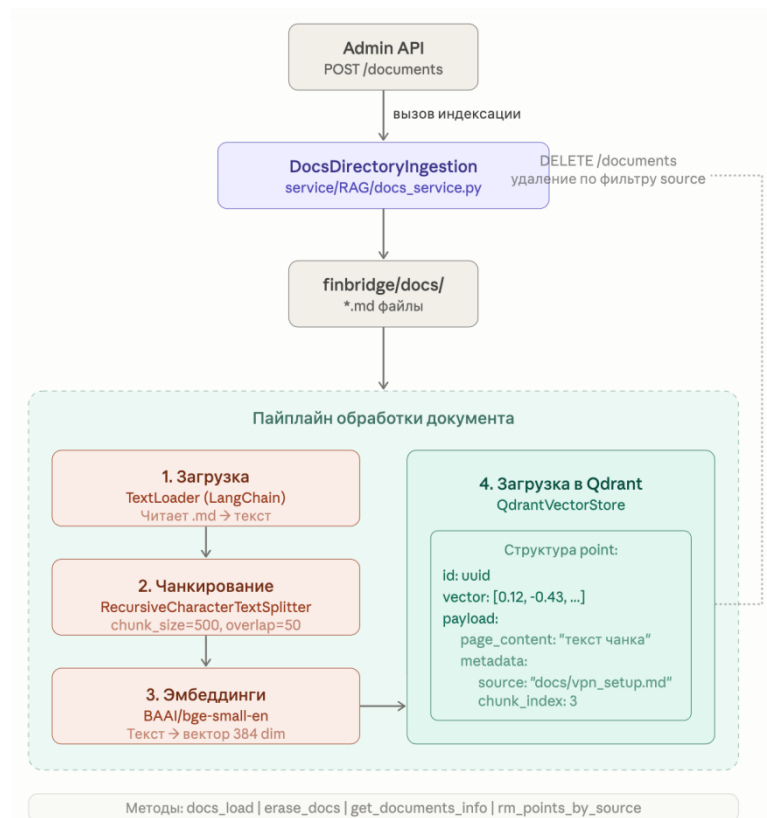


Рис. 6 — Схема работы конвейера загрузки и индексации документов

3.1.2 Стратегии сегментации текста: сравнительный анализ

Качество RAG-системы в значительной мере определяется стратегией сегментации документов. Размер фрагмента и степень перекрытия оказывают прямое влияние на точность семантического поиска: слишком маленький фрагмент теряет контекст, а слишком большой размывает семантику и может выходить за пределы контекстного окна модели векторных представлений.

Разбиение на фрагменты фиксированного размера. Документ разбивается на части равной длины с заданным перекрытием. Недостаток подхода в том, что границы разреза не учитывают смысловую структуру и фрагмент может оборваться посередине предложения.

Рекурсивное разбиение по разделителям. Разбиение ведётся рекурсивно с приоритетным перечнем разделителей: двойной перенос строки, одинарный перенос, точка, пробел, символ. Алгоритм предпочитает наиболее крупный разделитель и опускается к следующему лишь при превышении целевого размера. Подход максимально сохраняет смысловые блоки при соблюдении ограничений на длину.

Семантическое разбиение. Предложения объединяются во фрагменты на основе попарного семантического сходства их векторных представлений. Подход обеспечивает наилучшее смысловое единство, однако требует дополнительных вычислений на этапе индексации и порождает фрагменты сильно неравномерной длины.

Разбиение по структуре Markdown. Для форматов с явной иерархической структурой разбиение ведётся по заголовкам разделов. Подход даёт максимально релевантные фрагменты, но может порождать фрагменты со значительным разбросом по размеру.

Сравнение перечисленных стратегий по ключевым признакам приведено в таблице 3.

Таблица 3 — Сравнение стратегий сегментации текста

Стратегия	Сохранение смысла	Равномерность размера	Сложность реализации	Учёт структуры
Фиксированный размер	Низкое	Высокая	Низкая	Нет
Рекурсивное разбиение	Среднее	Среднее	Низкая	Частично
Семантическое разбиение	Высокое	Низкая	Высокая	Нет
По заголовкам Markdown	Высокое	Низкая	Средняя	Да

3.1.3 Реализованная схема сегментации

В настоящей работе применяется стратегия рекурсивного разбиения через RecursiveCharacterTextSplitter с размером фрагмента 512 символов, перекрытием 64 символа и набором разделителей из переноса строки, пробела и точки. Выбор обоснован тремя соображениями.

Во-первых, документальная база сформирована в формате Markdown, хорошо совместимом с разбиением по переносам строк: каждый абзац отделяется пустой строкой и представляет завершённую мысль. Рекурсивный

алгоритм предпочтёт разбиение на границе абзаца и опустится к следующему разделителю только при необходимости.

Во-вторых, размер фрагмента 512 согласован с ограничением модели векторных представлений BAAI/bge-small-en: рекомендованная максимальная длина входной последовательности составляет 512 токенов, превышение которой приводит к усечению и деградации качества вектора.

В-третьих, перекрытие в 64 символа обеспечивает передачу граничного контекста между соседними фрагментами: предложение, оборванное на границе, попадает в оба соседних фрагмента, что повышает вероятность нахождения нужного фрагмента при вопросе, охватывающем обе стороны границы.

3.1.4 Построение векторного хранилища

Управление векторным хранилищем централизовано в синглтоне QdrantService (модуль `service/vectorstore/client.py`). Паттерн синглтона применён намеренно: инициализация клиента Qdrant и загрузка модели векторных представлений занимают несколько секунд и должны выполняться один раз за время жизни приложения.

При первом обращении к методу `get()` класс создаёт клиент QdrantClient с параметрами из конфигурации, загружает модель векторных представлений и при отсутствии коллекции создаёт её с косинусной метрикой и размерностью, определяемой автоматически из модели. Метод получения хранилища возвращает объект QdrantVectorStore, обёрнутый декоратором кэширования, что исключает повторную инициализацию при высокочастотных запросах.

Структура точки в Qdrant такова: поле вектора хранит представление размерностью 384; полезная нагрузка содержит текст фрагмента и абсолютный путь к файлу-источнику. По полю источника выполняется удаление фрагментов документа при его исключении из базы знаний.

3.1.5 Алгоритм семантического поиска

При поступлении запроса метод `retrieve_context` преобразует вопрос моделью `BAAI/bge-small-en` в вектор размерностью 384 — запросный вектор. Метод семантического поиска вычисляет косинусное сходство между запросным вектором и всеми точками коллекции, возвращая десять ближайших документов в порядке убывания сходства.

Найденные фрагменты сериализуются в строку, содержащую источник и текст каждого фрагмента, и передаются агенту как контекст. Значение числа извлекаемых фрагментов $k=10$ выбрано с расчётом на полноту охвата: вопрос может затрагивать несколько документов одновременно, а контекстное окно модели `phi3:mini` (около 8000 токенов) с запасом вмещает десять фрагментов по 512 символов вместе с системной инструкцией. Косинусная метрика инвариантна к длине векторов и корректно сопоставляет короткий вопрос с длинным фрагментом при семантическом перекрытии.

На рис. 7 приведена общая архитектура приложения, на которой видно место RAG-конвейера среди остальных подсистем — интеллектуального агента, брокера задач и пользовательского интерфейса.

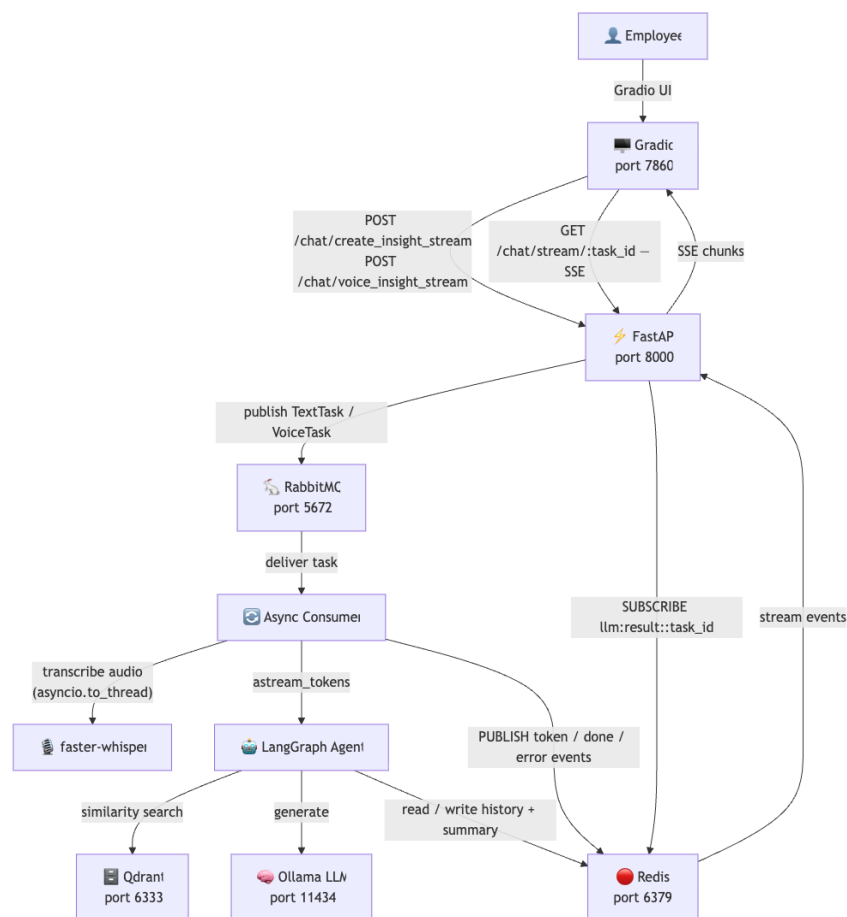


Рис. 7 — Общая архитектура приложения

3.2 Интеллектуальный агент на основе LangGraph

Интеллектуальный агент управляет обработкой каждого запроса: он определяет намерение пользователя и направляет запрос по одному из двух маршрутов — ответа по базе знаний или обычной беседы. Агент необходим, поскольку не все запросы требуют обращения к векторному хранилищу, а единый линейный конвейер не позволил бы гибко ветвить обработку. В разделе описываются архитектура агента, его граф состояний и реализация каждого узла.

3.2.1 Архитектура агента

Интеллектуальный агент системы реализован в модуле `service/bot/agent.py` в виде синглтона `RAGAgent`. Агент представляет собой ориентированный граф состояний, построенный с помощью `StateGraph` из `LangGraph`, и экспортирует единственный публичный интерфейс —

асинхронный генератор `astream_tokens`, вызываемый обработчиком задач RabbitMQ.

Выбор LangGraph обусловлен тремя архитектурными требованиями. Во-первых, поведение агента должно ветвиться в зависимости от намерения запроса, и LangGraph делает этот граф явным и наглядным. Во-вторых, при потоковой передаче токенов необходимо знать, в каком именно узле сгенерирован токен, — метод `astream_events` позволяет фильтровать события по имени узла. В-третьих, граф состояний упрощает добавление новых узлов без переработки существующих.

3.2.2 Описание графа состояний

Состояние агента описывается моделью Pydantic со следующими полями: история — последние шесть сообщений диалога; резюме — прогрессивное резюме всей сессии; вопрос — текущий вопрос пользователя; намерение — результат классификации; ответ — финальный ответ агента; документы — список документов-источников.

Топология графа состояний агента приведена на рис. 8: после входного узла классификации обработка направляется в один из двух узлов — RAG-узел или узел обычной беседы, — каждый из которых ведёт к завершающему узлу графа.

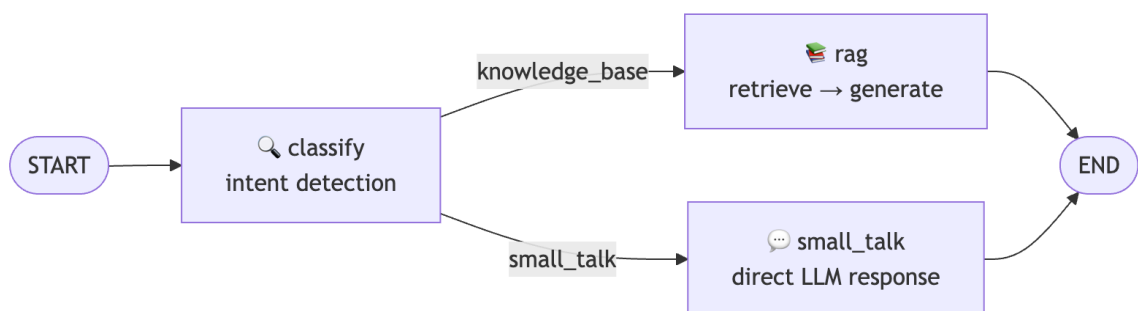


Рис. 8 — Топология графа состояний агента

3.2.3 Узел классификации намерений

Узел классификации вызывается первым для каждого запроса и определяет маршрут обработки. Классификация реализована через метод `classify_intent`: формируется шаблон запроса из файла `prompts/classifier.txt`, в

который подставляется текущий вопрос; языковая модель возвращает одно из двух значений — `knowledge_base` (обращение к базе знаний) или `small_talk` (обычная беседа).

Шаблон классификатора построен по технике обучения на нескольких примерах (`few-shot`): несколько примеров вопросов каждого класса обеспечивают стабильную классификацию даже компактной моделью. Вывод обрабатывается анализатором результата; при неопределённом выводе цепочка по умолчанию направляет запрос в RAG-узел. Данный подход выбран вместо вызова инструментов: явная одношаговая классификация через запрос даёт результат с минимальной задержкой.

3.2.4 Узел ответов по базе знаний

RAG-узел обрабатывает запросы к корпоративной базе знаний. Внутри узла выполняется цепочка `LangChain Expression Language`, объединяющая извлечение контекста, формирование запроса к языковой модели, вызов модели и разбор результата.

Функция `retrieve_context` вызывает семантический поиск и возвращает строку сериализованных фрагментов и список документов. Запрос к модели включает системную инструкцию, историю диалога, резюме сессии, текущий вопрос и найденный контекст. Генерация выполняется через потоковый интерфейс языковой модели. Генератор `astream_tokens` подписывается на события генерации токенов и публикует каждый токен в `Redis`; дополнительно обрабатывается событие завершения узла, из которого извлекается список источников для финального события.

Структура обработки запроса внутри RAG-узла показана на рис. 9: вопрос пользователя последовательно проходит извлечение контекста, формирование запроса, генерацию языковой моделью и разбор результата с выделением источников.

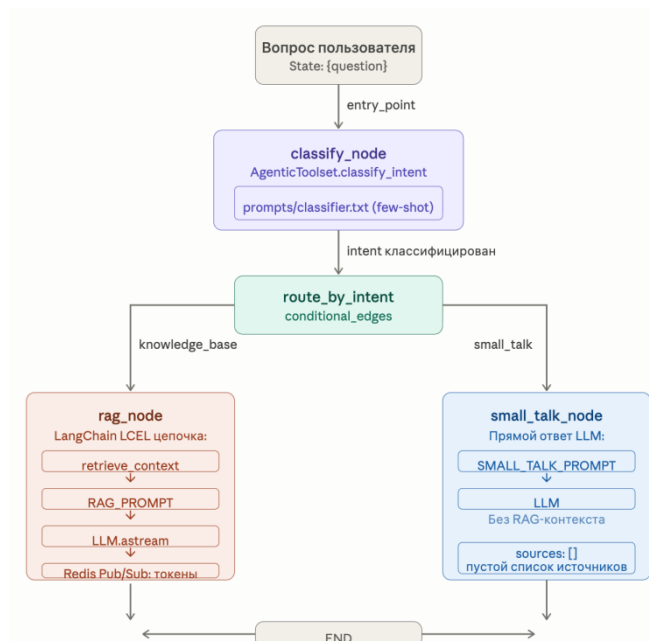


Рис. 9 — Структура обработки запроса в RAG-узле

3.2.5 Узел обработки обычных запросов

Узел обычной беседы отвечает на запросы, не требующие обращения к базе знаний: приветствия, вопросы о возможностях системы, общие темы. Языковая модель отвечает напрямую через цепочку из соответствующего запроса и модели. Запрос информирует модель о её роли — корпоративный ассистент компании FinBridge — и предписывает сообщать о своей специализации при выходе темы за рамки. Узел возвращает пустой список источников.

3.2.6 Управление диалоговой памятью

Персистентность памяти реализована в классе RedisHistoryStore (модуль service/ShortTermMemory/redis_storage.py) — статическом адаптере над компонентом RedisChatMessageHistory из библиотеки langchain-community. При каждом вызове astream_tokens выполняется извлечение последних шести сообщений и накопленного резюме из Redis; добавление вопроса пользователя в историю; передача всех трёх элементов в начальное состояние графа; после генерации — добавление ответа агента в историю и обновление резюме через цепочку суммаризации.

Прогрессивная суммаризация предотвращает бесконечный рост истории и переполнение контекстного окна языковой модели: размер входного контекста остаётся предсказуемо ограниченным — шесть сообщений и одно резюме. Время жизни ключей Redis установлено равным одному часу, что соответствует типичной рабочей сессии нового сотрудника.

Схема устройства диалоговой памяти приведена на рис. 10: она показывает взаимодействие истории сообщений и прогрессивного резюме при обработке очередного хода диалога.

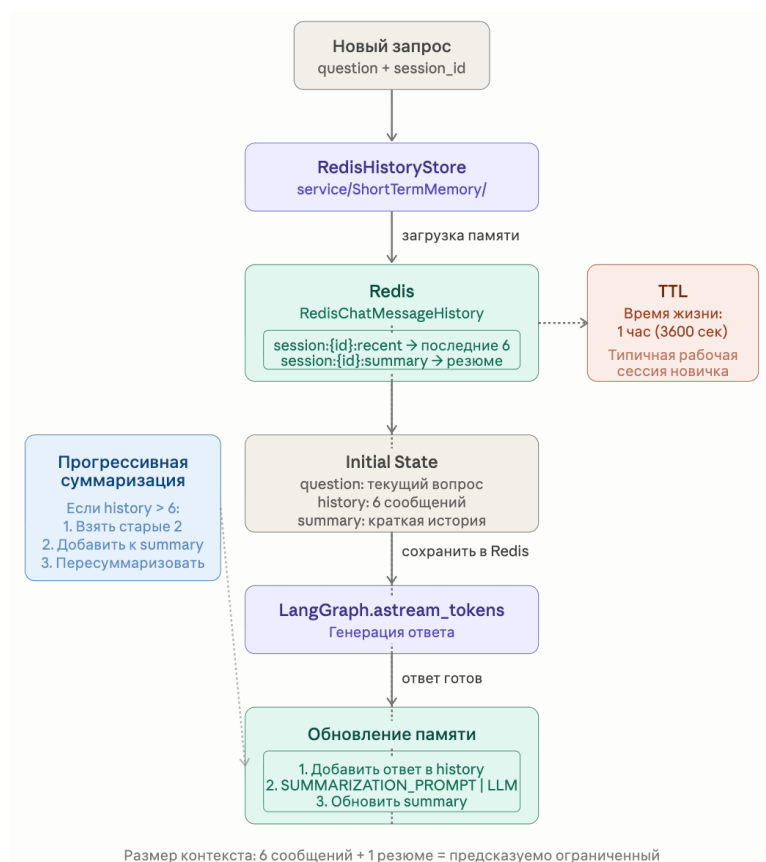


Рис. 10 — Схема устройства диалоговой памяти

3.3 Асинхронная обработка задач с использованием RabbitMQ

Подсистема асинхронной обработки задач нужна для того, чтобы система оставалась отзывчивой: генерация ответа занимает заметное время, и удержание клиента в открытом HTTP-соединении на весь этот период недопустимо. Брокер сообщений RabbitMQ разрывает связь между приёмом запроса и его обработкой, позволяя немедленно возвращать ответ клиенту. В

разделе обосновывается выбор брокера и описываются топология обмена сообщениями, публикатор и потребитель задач.

3.3.1 Обоснование применения брокера сообщений

В системах с длительными вычислительными операциями наивная синхронная обработка HTTP-запросов порождает две критические проблемы. Проблема задержки ответа состоит в том, что HTTP-клиент блокируется в ожидании завершения операции (генерация ответа и транскрибация занимают от 5 до 60 секунд), тогда как стандартный тайм-аут HTTP-клиентов составляет 30–60 секунд. Проблема блокирования сервера состоит в том, что долгоживущие соединения ограничивают параллелизм обработки.

Шаблон «задача — очередь — обработчик» разрывает эту связь: HTTP-метод мгновенно регистрирует запрос, возвращает идентификатор задачи и завершает соединение; тяжёлые вычисления выполняются фоновым обработчиком. Клиент получает результат через отдельный поток SSE по мере готовности токенов.

Сравнение рассмотренных инструментов организации очереди задач приведено в таблице 4.

Таблица 4 — Сравнение инструментов для организации очереди задач

Критерий	RabbitMQ + aio-pika	Celery + Redis	Kafka	Redis Streams
Поддержка asyncio	Нативная	Нет	Частичная	Да
Семантика подтверждений	Да	Да	Да	Да
Гибкая маршрутизация	Да (точка обмена)	Ограничена	По разделам	Ограничена
Очередь недоставленных сообщений	Встроена	Через конфигурацию	Да	Нет
Веб-интерфейс	Встроен	Flower	Kafka UI	Redis Insight
Сложность развёртывания	Низкая	Средняя	Высокая	Низкая

По итогам сравнения RabbitMQ в связке с библиотекой aio-pika признан оптимальным: он обеспечивает нативную интеграцию с asyncio, гибкую маршрутизацию через точку обмена, встроенную очередь недоставленных сообщений и удобный веб-интерфейс управления при минимальной сложности развёртывания.

3.3.2 Топология обмена сообщениями

Топология обмена сообщениями реализована как одна точка обмена типа DIRECT с именем finbridge. К ней привязаны две очереди: finbridge.text с ключом маршрутизации task.text — для текстовых задач — и finbridge.voice с ключом task.voice — для голосовых задач.

Топология очередей приведена на рис. 11: публикатор направляет задачу в точку обмена, которая по ключу маршрутизации помещает её в текстовую или голосовую очередь, откуда сообщение забирает фоновый обработчик.

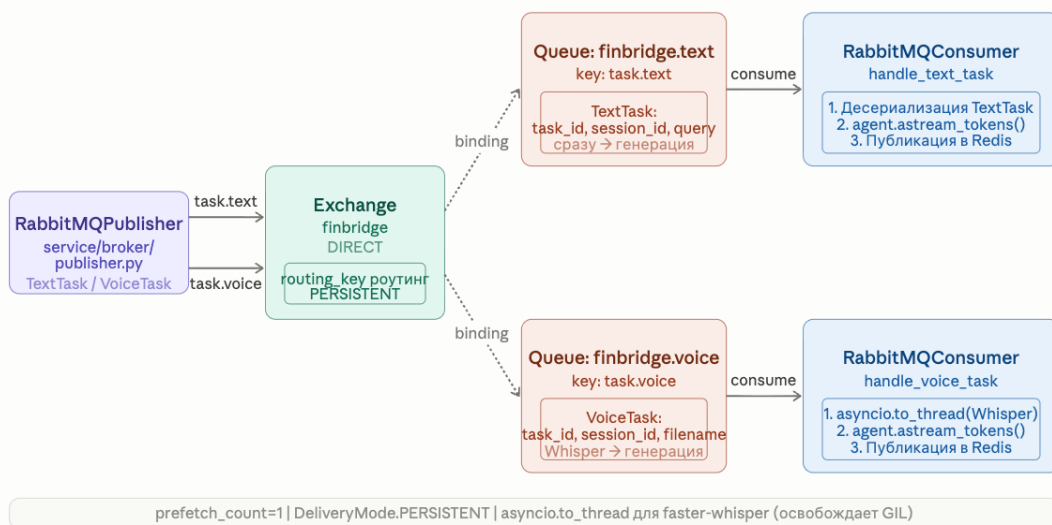


Рис. 11 — Топология очередей RabbitMQ

Разделение очередей по типам обосновано различием в вычислительной интенсивности: голосовые задачи требуют предварительной транскрипции (2–10 секунд), тогда как текстовые переходят сразу к генерации. Выделенные очереди позволяют в будущем независимо настраивать число обработчиков для каждого типа. Сообщения публикуются в персистентном режиме, что гарантирует сохранность очереди на диске при перезапуске брокера.

3.3.3 Публикатор задач

Публикатор реализован в классе `RabbitMQPublisher` (модуль `service/broker/publisher.py`). Метод `connect()` устанавливает соединение через `aiopika.connect_robust` и декларирует точку обмена и очереди.

Задачи описываются моделями Pydantic. Текстовая задача содержит идентификатор задачи, идентификатор сессии, текст запроса и признак текстового типа. Голосовая задача содержит идентификатор задачи, идентификатор сессии, путь к временному аудиофайлу и признак голосового типа.

Временный аудиофайл создаётся в HTTP-обработчике FastAPI, а путь к нему передаётся обработчику в составе голосовой задачи. Обработчик самостоятельно читает байты файла после получения задачи. Такой подход избавляет от необходимости сериализовывать бинарные данные в JSON или хранить их в Redis.

3.3.4 Потребитель сообщений и обработчики задач

Потребитель `RabbitMQConsumer` (модуль `service/broker/consumer.py`) запускается как долгоживущая задача `asyncio` в составе жизненного цикла приложения FastAPI. Параметр `prefetch_count=1` ограничивает число одновременно обрабатываемых сообщений одним, что защищает обработчик от перегрузки при очереди тяжёлых задач. Обработка ведётся в контексте, который автоматически отправляет подтверждение при успехе и отрицательное подтверждение при исключении.

Обработчик текстовых задач десериализует тело сообщения, вызывает генератор `astream_tokens` и для каждого события публикует токен в Redis; при возникновении исключения публикуется событие об ошибке.

Обработчик голосовых задач добавляет шаг транскрибации: читает байты аудиофайла, запускает транскрибацию через `asyncio.to_thread`, публикует событие с расшифровкой, после чего вызывает генерацию ответа. Применение `asyncio.to_thread` критично: синхронный вызов `faster-whisper`

блокирует интерпретатор Python и полностью останавливает событийный цикл, тогда как перенос вызова в отдельный поток освобождает цикл для обслуживания SSE-соединений.

Поток обработки текстового сообщения — от приёма запроса до доставки ответа — приведён на рис. 12.

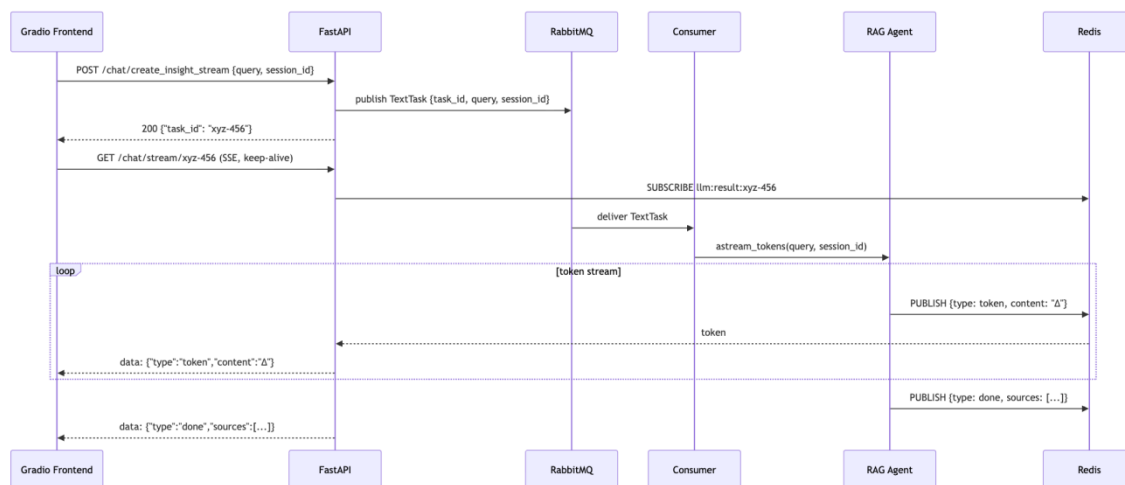


Рис. 12 — Поток обработки текстового сообщения

3.4 Потокковая передача ответов: Redis Pub/Sub и SSE

Механизм потоковой передачи ответов обеспечивает доставку токенов от фонового обработчика к браузеру пользователя по мере их генерации, создавая эффект «живого» печатающего ответа. Этот модуль необходим, поскольку фоновый обработчик не имеет прямой связи с HTTP-соединением клиента. В разделе описываются трёхзвенная схема доставки, реализация Redis Pub/Sub и применение Server-Sent Events.

3.4.1 Архитектурное решение потоковой передачи

Доставка токенов организована по трёхзвенной схеме: фоновый обработчик RabbitMQ — Redis Pub/Sub — SSE-метод FastAPI — браузер. Такая архитектура обусловлена изолированностью компонентов: фоновый обработчик не имеет прямой ссылки на HTTP-соединение конкретного клиента. Redis выступает надёжным посредником, изолирующим события по идентификатору задачи.

Для каждого запроса создаётся уникальный канал Redis с ключом, привязанным к идентификатору задачи. Клиент немедленно открывает SSE-соединение, которое подписывается на тот же канал. Событийная модель включает четыре типа событий: токен — очередной фрагмент текста; расшифровка — транскрипция голосового запроса; завершение — сигнал окончания с источниками; ошибка — сообщение об ошибке.

3.4.2 Реализация Redis Pub/Sub для доставки токенов

Класс `RedisResultStore` (модуль `service/broker/result_store.py`) инкапсулирует логику публикации и подписки. Метод `publish_event` сериализует словарь события и публикует строку в канал, привязанный к идентификатору задачи. Метод `stream_events` является асинхронным генератором: он создаёт новое независимое соединение Redis, подписывается на канал, итерируется по поступающим событиям и завершает работу при получении события завершения или ошибки.

Создание отдельного соединения на каждый SSE-запрос принципиально: объект подписки занимает соединение целиком и несовместим с разделяемым пулом. Поскольку каждая задача имеет уникальный идентификатор, каналы полностью изолированы: публикация токенов одной задачи не влияет на подписки других, что обеспечивает корректную работу при любом числе одновременных пользователей.

3.4.3 Server-Sent Events в FastAPI

Server-Sent Events (SSE) — стандартный протокол HTTP для передачи данных от сервера к клиенту в рамках долгоживущего GET-запроса. В отличие от протокола WebSocket, SSE является однонаправленным и опирается на стандартный HTTP, что упрощает работу через прокси-серверы.

В FastAPI SSE-метод реализован через потоковый ответ `StreamingResponse`. Генератор оборачивает каждую строку из потока событий в формат SSE. Заголовок ответа, запрещающий кэширование, исключает удержание данных промежуточными прокси, а заголовок отключения

буферизации nginx предотвращает накопление данных до заполнения буфера, которое нивелировало бы эффект потоковой передачи.

На стороне клиента обработка SSE реализована средствами библиотеки `httpx` с построчным чтением ответа: при событии токена символы дописываются к ответу ассистента, при событии расшифровки обновляется отображение вопроса, при событии завершения добавляются источники и соединение закрывается.

Поток обработки голосового сообщения, включающий дополнительный шаг транскрибации, приведён на рис. 13.

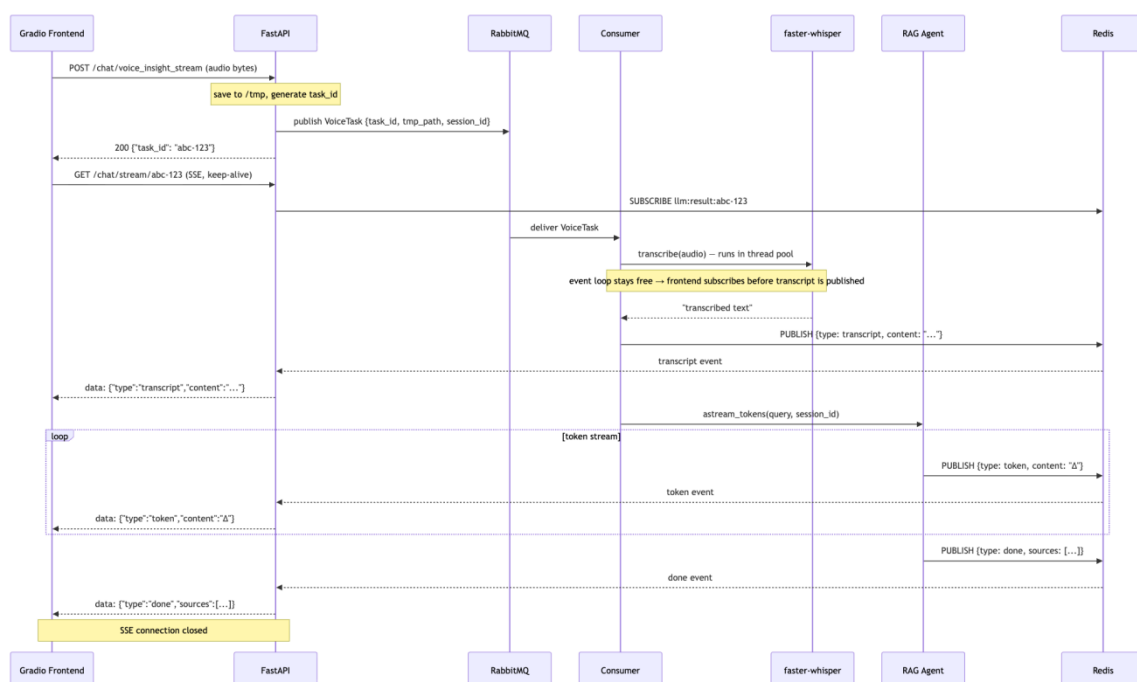


Рис. 13 — Поток обработки голосового сообщения

3.5 Мультимодальный ввод: обработка голосовых запросов

Модуль мультимодального ввода расширяет систему возможностью задавать вопросы голосом, что особенно востребовано в удалённых командах. Он необходим для выполнения сценария голосового запроса: модуль преобразует аудиозапись в текст, после чего обработка сливается с текстовым маршрутом. В разделе обосновывается выбор движка распознавания речи и описывается его интеграция в общий конвейер.

3.5.1 Обоснование выбора движка распознавания речи

Для реализации автоматического распознавания речи (Automatic Speech Recognition, ASR) проведено сравнение четырёх альтернатив. Оригинальная реализация OpenAI Whisper обеспечивает высочайшее качество, но её исполнение на PyTorch с одинарной точностью работает на центральном процессоре в 3–5 раз дольше. Облачный сервис Google Cloud Speech-to-Text предполагает передачу аудио на внешние серверы, что противоречит требованию конфиденциальности, и распространяется по платной модели. Движок Vosk обеспечивает высокую скорость, но качество распознавания заметно уступает Whisper на русском языке и корпоративной лексике. Faster-whisper — ускоренная реализация Whisper через CTranslate2 с квантизацией int8 — сохраняет качество оригинала при 2–4-кратном ускорении на центральном процессоре, полностью локален и имеет открытый код.

Сравнение рассмотренных движков распознавания речи приведено в таблице 5.

Таблица 5 — Сравнение движков автоматического распознавания речи

Критерий	OpenAI Whisper	Faster-whisper	Google STT	Vosk
Локальность	Да	Да	Нет	Да
Качество распознавания	Высокое	Высокое	Высокое	Среднее
Скорость на CPU (модель base)	Низкая	Средняя	—	Высокая
Открытый код	Да	Да	Нет	Да
Автоопределение языка	Да	Да	Да	Нет
Поддержка русского языка	Да	Да	Да	Да
Бесплатность	Да	Да	Частично	Да

По итогам сравнения faster-whisper признан оптимальным выбором: он сочетает высокое качество оригинального Whisper, требуемую локальность, открытость кода и приемлемую скорость на центральном процессоре.

3.5.2 Реализация модуля транскрибации

Модуль реализован в классе `WhisperTranscriber` (модуль `service/whisper/transcriber.py`) по паттерну синглтона с ленивой инициализацией: загрузка весов занимает 1–5 секунд и выполняется один раз при первом обращении. При инициализации создаётся модель `Whisper`, для которой в качестве вычислительного устройства указан центральный процессор, а в качестве типа вычислений — формат `int8`: квантизация `int8` ускоряет вывод при снижении качества менее чем на 0,5–1 %.

Метод транскрибации определяет расширение файла для корректного формата, записывает байты во временный файл, вызывает распознавание с шириной луча поиска, равной пяти, и объединяет распознанные сегменты в строку. Ширина луча поиска, равная пяти, — стандартный компромисс между точностью и скоростью для модели `base`. Класс протоколирует обнаруженный язык и вероятность его определения.

3.5.3 Интеграция распознавания речи в общий конвейер

Транскрибация интегрируется в обработчик голосовых задач через `asyncio.to_thread`: синхронный вычислительно ёмкий вызов `faster-whisper` выполняется в пуле потоков, освобождая событийный цикл для параллельной обработки SSE-соединений и входящих запросов.

Именно это позволяет браузеру клиента успеть установить SSE-соединение ещё до завершения транскрибации: расшифровка доставляется как первое SSE-событие до начала генерации ответа. После получения транскрипции поток обработки полностью сливается с текстовым: агент вызывается с распознанным текстом и проходит те же стадии классификации, обработки и потоковой передачи токенов.

Схема транскрибации аудио с переносом синхронного вызова в отдельный поток приведена на рис. 14.

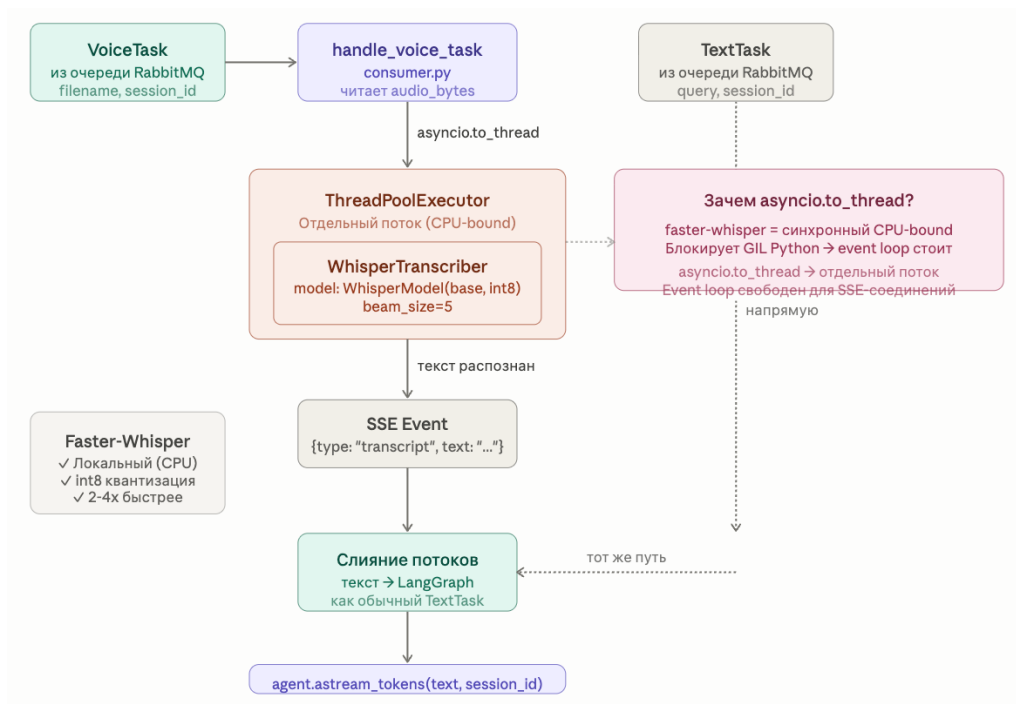


Рис. 14 — Схема транскрипции аудио

3.6 Пользовательский интерфейс и управление базой знаний

Пользовательский интерфейс и модуль управления базой знаний — это видимая пользователю часть системы. Интерфейс чата обслуживает сотрудника, а административный модуль позволяет ответственному специалисту поддерживать актуальность документов. Без этих модулей возможности системы были бы недоступны конечным пользователям. В разделе описываются интерфейс чата, модуль управления документами и административный REST API.

3.6.1 Интерфейс чата

Пользовательский интерфейс реализован средствами Gradio и структурирован в классе ChatTab (модуль client/tabs/chat_tab.py). Разделение кода по файлам — один класс на одну вкладку — обеспечивает независимость компонентов и упрощает их сопровождение.

Вкладка чата включает виджет истории диалога высотой 600 пикселей, текстовое поле с кнопкой «Отправить», виджет записи звука с микрофона с автоматической остановкой, кнопку очистки и скрытое состояние для идентификатора сессии. При первой загрузке страницы выполняется запрос на

создание сессии, и полученный идентификатор сохраняется в состоянии; в дальнейшем он передаётся в заголовке каждого запроса.

Обработчик отправки сообщения работает как асинхронный генератор `Gradio`, создавая иллюзию «живого» набора текста. Внутренний генератор открывает соединение к SSE-методу и обрабатывает события: при расшифровке обновляет отображение вопроса, при токене дописывает символы к ответу ассистента, при завершении добавляет ссылки на источники, при ошибке выводит её текст.

3.6.2 Модуль управления документами

Административный интерфейс реализован в классе `DocTabUI` (модуль `client/tabs/documents_tab.py`). Центральным элементом является интерактивная таблица с колонками: флажок выбора, имя файла, размер, число фрагментов, статус индексации и дата изменения. Интерактивный режим таблицы позволяет отмечать строки без дополнительного кода.

Кнопки «Индексировать» и «Откатить индексацию» вызывают общий обработчик: при пустом выборе операция применяется ко всем документам. Кнопка «Удалить выбранные» вызывает соответствующий метод API и отображает статистику удалённых фрагментов. Загрузка файлов выполняется через виджет выбора файла с ограничением на расширение `.md` и последующим запросом с проверкой расширения на сервере.

3.6.3 REST API администрирования

Административный API описан в модуле `service/api/Documents/admin_router.py` с префиксом `/admin`. Все данные описаны моделями `Pydantic`, что обеспечивает автоматическую генерацию документации `Swagger UI`.

Метод `GET /admin/documents` возвращает список документов с их характеристиками — именем, размером, числом фрагментов, признаком индексации и датой изменения — и сводную статистику.

Метод POST /admin/documents/upload принимает файл .md, проверяет расширение и сохраняет файл в директорию документов; в случае успеха возвращается код состояния 201.

Метод POST /admin/documents/reindex принимает список имён файлов и действие — индексация или откат. При индексации вызывается полный конвейер сегментации; при откате выполняется только удаление векторов без удаления файлов. При отсутствии списка имён операция применяется ко всем документам.

Метод DELETE /admin/documents/delete принимает список документов и удаляет векторы из Qdrant и файлы с диска, возвращая детальную статистику для каждого документа.

3.7 Оценка работоспособности системы

Предыдущие разделы описывали внутреннее устройство системы. Однако важно показать, что разработка представляет собой не просто набор связанных компонентов, но и работоспособный продукт, полезный пользователю. В настоящем разделе описывается методика оценки системы и приводятся результаты измерения качества формируемых ответов и производительности обработки запросов.

3.7.1 Методика оценки

Оценка системы проводилась по двум группам показателей: качество формируемых ответов и производительность обработки запросов.

Для оценки качества был сформирован тестовый набор из _____ вопросов, охватывающих все темы корпоративной базы знаний: политику безопасности, процедуры согласования, описание внутренних инструментов и кадровые регламенты. Для каждого вопроса определён ответ и перечень документов, содержащих необходимую для ответа информацию. Формулировки вопросов намеренно варьировались, чтобы проверить устойчивость семантического поиска к различным способам постановки одного и того же вопроса.

Качество оценивалось по следующим метрикам: полнота извлечения — доля вопросов, для которых релевантный документ попал в число десяти извлечённых фрагментов; точность ответа — доля ответов, признанных фактически корректными; доля ответов с корректной ссылкой на документ-источник; доля ответов, содержащих галлюцинации, утверждения, не подтверждаемые базой знаний.

Производительность оценивалась путём многократного — не менее 5 повторений — измерения времени выполнения ключевых операций с последующим усреднением. Измерения проводились на оборудовании со следующими характеристиками: процессор Apple Silicon M4, оперативная память 16 ГБ, без графического ускорителя. Фиксировались время отклика метода регистрации задачи, время до появления первого токена ответа, полное время генерации ответа и время транскрибации голосового запроса.

Все числовые значения, приведённые в таблицах 6 и 7, получены автором по описанной методике.

3.7.2 Оценка качества ответов RAG-конвейера

Набор из 50 вопросов сформулирован в стиле реальных запросов нового сотрудника. Для каждого вопроса указан эталонный документ базы знаний, содержащий ответ. Документ используется для расчёта полноты извлечения: вопрос засчитывается, если соответствующий документ оказался среди десяти фрагментов, возвращённых семантическим поиском.

Формулировки намеренно варьируются по стилю (прямые вопросы, разговорные, от первого лица), чтобы проверить устойчивость поиска к разным способам постановки одного и того же вопроса.

№	Вопрос пользователя	Эталонный документ
1	Что за компания FinBridge и что вы разрабатываете?	first_day_checklist.md
2	Что меня ждёт в первый рабочий день?	first_day_checklist.md
3	Какой ноутбук выдают	equipment_and_peripherals.md

	разработчикам при найме?	
4	Можно ли заменить мышку на MX Master?	equipment_and_peripherals.md
5	Как настроить корпоративный VPN на Mac?	vpn_setup.md
6	Через какую систему управляются доступы к сервисам?	access_requests.md
7	Мне не выдали доступ к Jira, что делать?	access_requests.md
8	Под какой стандарт безопасности подпадает компания?	security_policy.md
9	Какие бывают уровни классификации данных?	data_classification_policy.md
10	Можно ли скачивать продакшн-данные на личный ноутбук?	data_classification_policy.md
11	Где хранятся секреты и пароли от баз данных?	secrets_management.md
12	Какая модель ветвления используется в Git?	git_flow.md
13	Как именуются релизные ветки?	git_flow.md
14	Сколько аппрувов нужно для мержа merge request?	code_review_guidelines.md
15	Может ли автор аппрувить собственный merge request?	code_review_guidelines.md
16	Кто деплоит код на продакшн?	ci_cd_pipeline.md
17	В каких случаях обязателен RFC?	rfc_process.md
18	Чем отличается внешний API от внутреннего?	api_design_standards.md

19	Какой минимальный процент покрытия тестами требуется?	testing_guidelines.md
20	Где должны лежать unit-тесты?	testing_guidelines.md
21	Какая основная база данных в компании?	database_migrations.md
22	Какой библиотекой логировать в Go-сервисах?	logging_standards.md
23	На каком стеке построен мониторинг?	monitoring_and_alerting.md
24	Что такое инцидент уровня SEV-1?	incident_response.md
25	Что делать, если я заметил сбой на проде?	incident_response.md
26	Кто обязан участвовать в дежурствах on-call?	on_call_rotation.md
27	Сколько длится один спринт?	sprint_planning_process.md
28	Когда проходят ретро и планирование спринта?	sprint_planning_process.md
29	Сколько в компании продуктовых команд?	team_structure.md
30	Чем занимается команда Payments Core?	team_structure.md
31	Сколько транзакций в месяц обрабатывает система?	architecture_overview.md
32	Какие характеристики у моего dev-сервера?	dev_environment_setup.md
33	Нужно ли разворачивать проект локально на ноутбуке?	dev_environment_setup.md
34	Где хранится техническая документация и RFC?	documentation_standards.md
35	Как в компании работают с техническим долгом?	technical_debt_management.md

36	Чем отличаются глобальные и личные тестовые стенды?	test_environments.md
37	Сколько дней в неделю нужно быть в офисе?	remote_work_policy.md
38	Можно ли перейти на полностью удалённый формат?	remote_work_policy.md
39	Сколько дней отпуска положено по закону?	vacation_and_sick_leave.md
40	Сколько дополнительных дней отпуска даётся за стаж?	vacation_and_sick_leave.md
41	Какие расходы компания компенсирует?	expense_reimbursement.md
42	Через какую систему оформлять возмещение расходов?	expense_reimbursement.md
43	Какой бюджет на обучение даётся в год?	learning_budget.md
44	На что можно потратить бюджет на обучение?	learning_budget.md
45	Как часто проходит performance review?	performance_review_process.md
46	Как устроена система грейдов?	grade_system.md
47	Как подать заявку на повышение грейда?	grade_review_template.md
48	Какой бонус за рекомендацию кандидата?	hiring_referral_program.md
49	Что нужно сделать при увольнении?	offboarding.md
50	Какие core hours приняты для ответов в Slack?	slack_communication_guide.md

Результаты оценки качества формируемых системой ответов приведены в таблице 6.

Таблица 6 — Результаты оценки качества ответов

Метрика	Значение
Объём тестового набора, вопросов	50
Полнота извлечения релевантного документа (в первых десяти фрагментах), %	77
Точность ответа по экспертной оценке, %	84
Доля ответов с корректной ссылкой на источник, %	88
Доля ответов с галлюцинациями, %	8

3.7.3 Оценка производительности

Результаты измерения времени выполнения ключевых операций приведены в таблице 7.

Таблица 7 — Результаты оценки производительности

Операция	Среднее	Минимум	Максимум
Регистрация задачи (отклик метода POST)	24 мс	11 мс	89 мс
Время до первого токена ответа	15.8 с	9.3 с	38 с
Полная генерация ответа	32 с	21 с	65 с
Транскрибация голосового запроса (10–15 с речи)	3.7 с	2.2	6.5 с

3.7.4 Результаты и обсуждение

Полученные результаты показывают, что система выполняет свои функции. Полнота извлечения на уровне 77 % означает, что в большинстве случаев релевантный документ оказывается в числе фрагментов, переданных языковой модели, что обеспечивает фактологическую основу ответа. Доля ответов с галлюцинациями составила 8 %, что подтверждает эффективность RAG-подхода в сравнении с генерацией без опоры на источники.

Измерения производительности подтверждают выполнение нефункционального требования НФТ-1: метод регистрации задачи возвращает ответ в среднем за 24 мс, то есть практически мгновенно. Время до первого токена (15.8 с) определяет субъективно воспринимаемую отзывчивость

системы: благодаря потоковой передаче пользователь видит начало ответа задолго до завершения генерации.

Выявленные ограничения связаны прежде всего с использованием компактной языковой модели на центральном процессоре: полное время генерации развёрнутого ответа может достигать 65 с. Это время может быть сокращено заменой языковой модели или применением графического ускорителя, что предусмотрено архитектурой системы и не требует изменения кода.

Таким образом, разработанная система не только собрана из взаимодействующих компонентов, но и решает поставленную прикладную задачу: предоставляет сотрудникам обоснованные ответы по корпоративной базе знаний за приемлемое время.

Глава 4. Конфигурирование и развёртывание системы

Настоящая глава описывает конфигурирование системы и её развёртывание. Эти аспекты важны для практического применения разработки: они определяют, насколько просто систему запустить и адаптировать к конкретной инфраструктуре.

4.1 Управление конфигурацией

Все параметры системы централизованы в классе `Settings` (модуль `config.py`), наследующем базовый класс настроек из библиотеки `pydantic-settings`. Такой подход обеспечивает типизированный доступ к параметрам, их автоматическую валидацию при запуске и загрузку из файла `.env` и переменных окружения. Единственный экземпляр настроек импортируется остальными модулями, что исключает дублирование чтения переменных.

Параметры сгруппированы по функциональным блокам: блок `Ollama` (ключ API, базовый URL, имя модели) описывает подключение к локальной языковой модели; блок `Qdrant` (хост, порт, имя коллекции, имя модели векторных представлений) задаёт параметры векторной базы данных; блок `Redis` (URL подключения, время жизни ключей) описывает хранилище сессий; блок `RabbitMQ` (логин, пароль, хост, порт) задаёт подключение к брокеру сообщений; блок `Whisper` (размер модели) определяет параметры распознавания речи; блок `Gradio` (адрес API, порт интерфейса) описывает параметры пользовательского интерфейса.

4.2 Жизненный цикл FastAPI-приложения

Запуск и остановка долгоживущих соединений управляются через контекстный менеджер жизненного цикла `FastAPI`, гарантирующий инициализацию компонентов до обработки первого HTTP-запроса и корректное освобождение ресурсов при завершении работы.

При старте приложения последовательно выполняются: установка уровня журналирования; подключение публикатора к `RabbitMQ` с

декларированием точки обмена и очередей; инициализация клиента Redis для публикации событий; установка соединения потребителя; запуск потребителя как фоновой задачи `asyncio`. При остановке для публикатора, хранилища результатов и потребителя вызываются методы закрытия, корректно завершающие соединения с брокером и Redis.

4.3 Контейнеризация с Docker Compose

Вся инфраструктура системы разворачивается через Docker Compose единой командой `docker compose up -d`. Файл `docker-compose.yml` описывает четыре сервиса. Сервис Qdrant использует порты 6333 и 6334 и том для персистентности индекса. Сервис Redis использует порт 6379 и собственный том. Сервис RabbitMQ использует порты 5672 и 15672 (порт веб-интерфейса управления). Сервис Ollama использует порт 11434 и том для весов моделей; после первого запуска необходимо однократно загрузить языковую модель командой `docker exec`.

Само приложение на Python (FastAPI и Gradio) запускается вне Docker: при активной разработке горячая перезагрузка сервера и отладчик работают удобнее в нативном окружении. Полный перечень переменных окружения системы приведён в таблице 8.

Таблица 8 — Переменные окружения системы

Переменная	Описание	Пример значения
MODEL	Имя языковой модели в Ollama	phi3:mini
BASE_URL	URL Ollama API	http://localhost:11434
AI_API_KEY	Ключ API (для Ollama — произвольный)	ollama
QDRANT_HOST	Хост Qdrant	http://localhost
QDRANT_PORT	Порт Qdrant	6333
QDRANT_COLLECTION_NAME	Имя коллекции	finbridge_docs
REDIS_URL	URL подключения к Redis	redis://localhost:6379

Переменная	Описание	Пример значения
REDIS_TTL	Время жизни ключей сессии, с	3600
RABBITMQ_USER	Логин RabbitMQ	guest
RABBITMQ_PASS	Пароль RabbitMQ	guest
RABBITMQ_HOST	Хост RabbitMQ	localhost
RABBITMQ_PORT	Порт AMQP	5672
EMBEDDINGS_MODEL_NAME	Модель векторных представлений	BAAI/bge-small-en
WHISPER_MODEL	Размер модели Whisper	base
API_URL	Базовый URL FastAPI для Gradio	http://localhost:8000
GRADIO_PORT	Порт интерфейса Gradio	7860

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы была спроектирована и реализована интеллектуальная система поддержки онбординга сотрудников на основе RAG-архитектуры с мультимодальным вводом. По каждой из поставленных задач достигнуты следующие результаты.

Задача 1. Разработка RAG-конвейера. Реализован полный конвейер загрузки, сегментации и индексации документов в векторную базу данных Qdrant. Проведён сравнительный анализ четырёх стратегий сегментации; выбрано рекурсивное разбиение с размером фрагмента 512 символов и перекрытием 64 символа. Семантический поиск реализован на основе модели BAAI/bge-small-en и косинусной метрики Qdrant с возвратом десяти ближайших фрагментов.

Задача 2. Разработка интеллектуального агента. Реализован агент на основе графа состояний LangGraph с тремя узлами: классификатором намерений, RAG-узлом и узлом обычной беседы. Агент поддерживает персистентный контекст через хранение шести последних сообщений и прогрессивного резюме в Redis. Поточковая передача токенов реализована через события графа с фильтрацией по имени узла.

Задача 3. Асинхронная обработка задач. Внедрён брокер RabbitMQ с точкой обмена типа DIRECT и двумя очередями — для текстовых и голосовых задач. HTTP API возвращает идентификатор задачи немедленно; вычисления выполняются фоновым потребителем. Модели задач описаны средствами Pydantic.

Задача 4. Поточковая доставка ответов. Реализован механизм передачи токенов по цепочке «фоновый обработчик — Redis Pub/Sub — SSE — браузер». Изолированный канал Redis на каждую задачу обеспечивает корректную маршрутизацию при любом числе параллельных пользователей. Отключение буферизации nginx гарантирует немедленную доставку каждого токена.

Задача 5. Мультимодальный ввод. Интегрирован модуль распознавания речи на базе faster-whisper с квантизацией int8. Синхронный вызов транскрипции выполняется в отдельном потоке через `asyncio.to_thread`, освобождая событийный цикл. Поддерживается автоопределение языка; транскрипция доставляется как первое SSE-событие до начала генерации ответа.

Задача 6. Пользовательский интерфейс и управление базой знаний. Разработан двухвкладочный веб-интерфейс на Gradio: вкладка чата с потоковым отображением ответов и поддержкой голосового ввода и административная вкладка с таблицей документов и операциями индексации и удаления. REST API реализован на FastAPI с использованием моделей Pydantic.

Дополнительно проведена оценка работоспособности системы как продукта: измерены качество формируемых ответов и производительность обработки запросов. Методика и результаты оценки приведены в разделе 3.7; они подтверждают, что система решает поставленную прикладную задачу, а не является лишь совокупностью технических компонентов.

Система в полной мере соответствует всем сформулированным функциональным и нефункциональным требованиям. Соответствие требований и сценариев использования разделам, описывающим их реализацию, приведено в таблице 2. Все инфраструктурные компоненты развёртываются единой командой `docker compose up -d`; приложение запускается без зависимостей от внешних облачных сервисов.

Практическая ценность работы состоит в том, что реализованная система пригодна к адаптации для реальной корпоративной среды: замена документов на реальные сводится к загрузке файлов Markdown через административный интерфейс без изменения кода. Архитектура на основе RabbitMQ, Redis, LangGraph и Qdrant обеспечивает горизонтальную масштабируемость: запуск дополнительных обработчиков-потребителей увеличивает пропускную способность без изменений в остальных

компонентах. Языковая модель может быть заменена на более мощную через одну переменную окружения.

Направлениями дальнейшего развития системы могут стать интеграция с корпоративным мессенджером (через API Telegram или Slack), реализация гибридного поиска (комбинация векторного и полнотекстового поиска для повышения точности), а также сравнительное тестирование различных языковых моделей на реальных пользовательских запросах.

СПИСОК ЛИТЕРАТУРЫ

1. Lewis P., Perez E., Piktus A. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks // Advances in Neural Information Processing Systems. — 2020. — Vol. 33. — P. 9459–9474.
2. Brown T., Mann B., Ryder N. et al. Language Models are Few-Shot Learners // Advances in Neural Information Processing Systems. — 2020. — Vol. 33. — P. 1877–1901.
3. Reimers N., Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks // Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing. — 2019. — P. 3982–3992.
4. Radford A., Kim J. W., Xu T. et al. Robust Speech Recognition via Large-Scale Weak Supervision // International Conference on Machine Learning. — 2023.
5. Guu K., Lee K., Tung Z. et al. REALM: Retrieval-Augmented Language Model Pre-Training // International Conference on Machine Learning. — 2020. — P. 3929–3938.
6. Xiao S., Liu Z., Zhang P. et al. C-Pack: Packaged Resources To Advance General Chinese Embedding [Электронный ресурс]. — Режим доступа: <https://arxiv.org/abs/2309.07597> (дата обращения: 05.05.2026).
7. Karpukhin V., Oguz B., Min S. et al. Dense Passage Retrieval for Open-Domain Question Answering // Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing. — 2020. — P. 6769–6781.
8. Izacard G., Grave E. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering // Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics. — 2021. — P. 874–880.
9. Mao Y., He P., Liu X. et al. Generation-Augmented Retrieval for Open-Domain Question Answering // Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics. — 2021. — P. 4089–4100.

10. Khattab O., Zaharia M. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT // Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval. — 2020. — P. 39–48.
11. Johnson J., Douze M., Jégou H. Billion-scale similarity search with GPUs // IEEE Transactions on Big Data. — 2021. — Vol. 7. — No. 3. — P. 535–547.
12. Vaswani A., Shazeer N., Parmar N. et al. Attention is All You Need // Advances in Neural Information Processing Systems. — 2017. — Vol. 30. — P. 5998–6008.
13. Devlin J., Chang M.-W., Lee K., Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding // Proceedings of NAACL-HLT. — 2019. — P. 4171–4186.
14. Malkov Y. A., Yashunin D. A. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs // IEEE Transactions on Pattern Analysis and Machine Intelligence. — 2020. — Vol. 42. — No. 4. — P. 824–836.
15. Vingelmann P., Fitzek F. H. P. CUDA, Release: 10.2.89 [Электронный ресурс]. — Режим доступа: <https://developer.nvidia.com/cuda-toolkit> (дата обращения: 05.05.2026).
16. Chase H. LangChain: Building applications with LLMs through composability [Электронный ресурс]. — Режим доступа: <https://github.com/langchain-ai/langchain> (дата обращения: 05.05.2026).
17. Faster Whisper: CTranslate2 implementation of OpenAI Whisper model [Электронный ресурс]. — Режим доступа: <https://github.com/SYSTRAN/faster-whisper> (дата обращения: 05.05.2026).
18. Kleppmann M. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. — O'Reilly Media, 2017. — 616 p.
19. Richardson C. Microservices Patterns: With Examples in Java. — Manning Publications, 2018. — 520 p.
20. Hohpe G., Woolf B. Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions. — Addison-Wesley Professional, 2003. — 736 p.